

Evaluation of Text Representation Methods for Iconclass Label Recommendation

Bachelor Thesis

by

Beril Tezel

Degree Course: Industrial Engineering and Management B.Sc.

Matriculation Number: 2327437

Institute of Applied Informatics and Formal Description
Methods (AIFB)

KIT Department of Economics and Management

Advisor: Prof. Dr. Harald Sack

Second Advisor: Dr.-Ing. Tobias Käfer

Supervisor: Etienne Posthumus, Tabea Tietz

Submitted: September 4, 2026

Zusammenfassung

Digitale Sammlungen des kulturellen Erbes erfordern wirksame Suchverfahren, um die vielfältigen Queries der Nutzerinnen und Nutzer mit strukturierten Klassifikationsvokabularen zu verknüpfen. Iconclass stellt hierarchische, standardisierte Labels zur Beschreibung von Bildinhalten bereit, jedoch können Unterschiede zwischen der Formulierung der Queries und der Iconclass-Terminologie klassische Retrieval-Verfahren wie BM25 in ihrer Leistungsfähigkeit einschränken. Semantische Embedding-Verfahren könnten diese Einschränkungen überwinden, ihre Eignung für die Empfehlung von Iconclass-Labels wurde bislang jedoch nicht systematisch untersucht.

Die vorliegende Arbeit vergleicht BM25, SBERT, EmbeddingGemma und den Text Encoder von SigLIP2 hinsichtlich des Retrievals relevanter Iconclass-Notationen anhand textueller Queries. Ihre Ergebnisse wurden auf derselben Collection gegenüber einer manuell erstellten Ground Truth evaluiert. Der Vergleich zeigte, dass kein einzelnes Verfahren die übrigen durchgängig übertraf. Das lexikalische Matching war am stärksten, wenn die Query-Terme in den Labels vorkamen, während Embedding-basierte Verfahren besser abschnitten, wenn dieselben Konzepte anders ausgedrückt wurden. SBERT und EmbeddingGemma lieferten sehr ähnliche Ergebnisse, während SigLIP2 insgesamt am schwächsten war, bei szenenbasierten Queries jedoch am besten abschnitt.

Die Ergebnisse zeigen darüber hinaus, dass die hierarchische Struktur von Iconclass bei der Evaluation berücksichtigt werden muss, da sich viele vermeintliche Fehler als knappe Fehltreffer innerhalb der korrekten Region erwiesen. Insgesamt legen die Ergebnisse eine Kombination lexikalischer und Embedding-basierter Verfahren nahe, da ihre komplementären Stärken sie für unterschiedliche Query-Typen jeweils besser geeignet machen. Die Befunde unterstreichen zudem, wie wichtig es ist, die hierarchische Struktur von Iconclass bei der Bewertung der Retrieval-Performance einzubeziehen. Digitale Sammlungen des kulturellen Erbes erfordern wirksame Suchverfahren, um die vielfältigen Queries der Nutzerinnen und Nutzer mit strukturierten Klassifikationsvokabularen zu verknüpfen. Iconclass stellt hierarchische, standardisierte Labels zur Beschreibung von Bildinhalten bereit, jedoch können Unterschiede zwischen der Formulierung der Queries und der Iconclass-Terminologie klassische Retrieval-Verfahren wie BM25 in ihrer Leistungsfähigkeit einschränken. Semantische Embedding-Verfahren könnten diese Einschränkungen überwinden, ihre Eignung für die Empfehlung von Iconclass-Labels wurde bislang jedoch nicht system-

atisch untersucht.

Die vorliegende Arbeit vergleicht BM25, SBERT, EmbeddingGemma und den Text Encoder von SigLIP2 hinsichtlich des Retrievals relevanter Iconclass-Notationen anhand textueller Queries. Ihre Ergebnisse wurden auf derselben Collection gegenüber einer manuell erstellten Ground Truth evaluiert. Der Vergleich zeigte, dass kein einzelnes Verfahren die übrigen durchgängig übertraf. Das lexikalische Matching war am stärksten, wenn die Query-Terme in den Labels vorkamen, während Embedding-basierte Verfahren besser abschnitten, wenn dieselben Konzepte anders ausgedrückt wurden. SBERT und EmbeddingGemma lieferten sehr ähnliche Ergebnisse, während SigLIP2 insgesamt am schwächsten war, bei szenenbasierten Queries jedoch am besten abschnitt.

Die Ergebnisse zeigen darüber hinaus, dass die hierarchische Struktur von Iconclass bei der Evaluation berücksichtigt werden muss, da sich viele vermeintliche Fehler als knappe Fehltreffer innerhalb der korrekten Region erwiesen. Insgesamt legen die Ergebnisse eine Kombination lexikalischer und Embedding-basierter Verfahren nahe, da ihre komplementären Stärken sie für unterschiedliche Query-Typen jeweils besser geeignet machen.

Abstract

Digital cultural heritage collections require effective search methods to connect users' varied queries with structured classification vocabularies. Iconclass provides hierarchical, standardized labels for describing visual content, but differences between query wording and Iconclass terminology can limit traditional retrieval methods such as BM25. Semantic embedding methods may overcome these limitations, but their suitability for Iconclass label recommendation has not yet been systematically investigated.

This thesis compares BM25, SBERT, EmbeddingGemma and the text encoder of SigLIP2 for retrieving relevant Iconclass notations from textual queries. Their results were evaluated on the same collection against a manually constructed ground truth. The comparison showed that no single method consistently outperformed the others. Lexical matching was strongest when query terms appeared in the labels, whereas embedding-based methods performed better when the same concepts were expressed differently. SBERT and EmbeddingGemma produced very similar results, while SigLIP2 was weakest overall but performed best for scene-based queries.

The findings further show that Iconclass's hierarchical structure needs to be considered in evaluation, as many apparent errors were near misses within the correct region. Overall, the results suggest combining lexical and embedding-based methods, as their complementary strengths make them better suited to different types of queries.

Contents

1	Introduction	6
1.1	Research Questions	7
1.2	Objective	7
1.3	Structure of the Thesis	8
2	Background	8
2.1	Iconclass	8
2.2	The Nature of Retrieval in Iconclass	11
2.3	Ground Truth	11
3	Theoretical Foundations	12
3.1	Representing and Comparing Texts	13
3.1.1	Vector Embeddings	13
3.1.2	Encoding a Text as a Single Vector	13
3.1.3	Cosine Similarity	14
3.2	Text Representation Methods	15
3.2.1	Best Matching 25	15
3.2.2	SBERT	16
3.2.3	EmbeddingGemma	16
3.2.4	SigLIP 2	17
4	Methodology	18
4.1	Preparation of the Data	19
4.1.1	Extraction of the Iconclass Data	19
4.1.2	Construction of the Search Collection	19
4.1.3	Iconclass Database	19
4.2	Construction of Ground Truth	20
4.3	Implementation of the Methods	21
4.3.1	BM25	21
4.3.2	SBERT	21

4.3.3	EmbeddingGemma	22
4.3.4	SigLIP 2	22
4.4	Evaluation Metrics	23
4.4.1	Precision	23
4.4.2	Recall	24
4.4.3	F_1 -Score	24
4.4.4	R-Precision	25
4.4.5	Mean Average Precision (MAP)	25
4.4.6	Wu-Palmer Similarity	26
5	Results	28
6	Discussion	33
7	Conclusion	38

1 Introduction

As digital cultural heritage collections continue to grow, the question of how users can find relevant content becomes more important. Digitization makes these materials available to a wider audience, but this accessibility also depends on the quality of the search results users receive. Search in this context can be difficult, since users may describe what they are looking for in different ways and with different levels of detail. This requires bridging the gap between users’ everyday language and the structured vocabulary used to describe content. At the same time, relevance in this type of search is not always straightforward, as a query can be related to broader, narrower, or contextually connected concepts. In this context, structured classification systems play a central role in improving access to cultural heritage collections by organizing data and supporting information retrieval.

One widely used system for classifying visual content is Iconclass, which is used in several museum, library, and research databases for subject access to image collections [2; 20].¹ It is especially relevant for professional cultural heritage catalogers and researchers, who search within the system to identify suitable categories for describing cultural heritage material. Iconclass provides a hierarchical way of describing subjects in works of art through alphanumeric notations [20]. Each of these codes is linked to a textual description of a concept, here referred to as a notation. This system allows the artworks to be indexed in a consistent and standardized way. By assigning the same notation and corresponding label to similar content, different institutions can describe artworks in a comparable manner. At the same time, however, this structure also introduces challenges for information retrieval.

A key difficulty lies in the gap between the wording used by catalogers when searching for a subject category and the vocabulary used in Iconclass labels. Even professional users who are familiar with controlled vocabularies may formulate a search phrase differently from the wording used in the system. Traditional retrieval approaches, such as BM25, score documents based on the presence and frequency of query terms within them and therefore perform best when the terms used in the query also appear in the document [25]. As a result, these approaches may miss relevant labels or rank them too low when the wording of the query differs from the wording used in Iconclass. For example, a user searching for “woman holding a baby” may not retrieve a label described as “Madonna and Child”, since the two descriptions refer to a similar visual concept but through very different vocabulary.

To address these limitations, more recent approaches from natural language processing offer promising alternatives. Embedding-based methods, such as SBERT, consider se-

¹On the use of Iconclass by museums, libraries and documentation institutes see <https://iconclass.org/help/about>.

mantic similarity beyond exact term overlap. They have shown strong performance in search and ranking tasks, where a query needs to be matched with the most relevant text [24]. Beyond SBERT, more recent text embedding models and text encoders from vision-language models offer further possibilities for matching queries with labels. However, to the best of my knowledge, it has never been researched how these different text representation methods perform in the specific context of Iconclass label recommendation. This task differs from general retrieval settings because Iconclass combines short, domain-specific textual labels with a large and hierarchically organized classification structure. It therefore remains unclear which type of text representation is most suitable for matching textual queries to Iconclass labels, and how these methods differ in their strengths and limitations.

1.1 Research Questions

This thesis aims to answer the following main research question:

RQ1 How do BM25, SBERT, EmbeddingGemma, and SigLIP 2 compare for recommending ICONCLASS labels from short textual queries?

This main question is further addressed through the following subquestions:

RQ1.1 To what extent do the embedding-based methods SBERT, EmbeddingGemma, and SigLIP 2 outperform the lexical baseline, BM25, in ICONCLASS label retrieval?

RQ1.2 To what extent does the SigLIP 2 text encoder differ in retrieval performance from text-only embedding models (SBERT, EmbeddingGemma)?

RQ1.3 To what extent are the retrieved notations and the ground-truth notations hierarchically related according to Wu-Palmer similarity?

1.2 Objective

To answer the research questions, this thesis evaluates four models: BM25, SBERT, EmbeddingGemma, and the text encoder of SigLIP 2. BM25 is included as an exact keyword-matching method, while SBERT and EmbeddingGemma represent embedding-based approaches. SigLIP 2 is included through its text encoder as a vision-language model that provides another form of text representation. The methods are applied to textual search queries and used to retrieve ranked Iconclass labels. Their results are then compared against a manually defined ground truth using standard information retrieval metrics. All code used for the data preparation, the four retrieval methods and the

evaluation, together with the ground truth and the result files, is publicly available to make the evaluation reproducible.²

1.3 Structure of the Thesis

This thesis is structured as follows. In Section 2, background information is provided to understand the remainder of the thesis better. Section 3 introduces the theoretical foundations of the natural language processing methods addressed in this thesis and their related work. Section 4 outlines the dataset preparation, application of the selected methods, and criteria used to compare their performance. This is followed by section 5, in which the results are displayed. In section 6, the findings are interpreted and the strengths and limitations of each approach are highlighted. Section 7 then concludes the thesis by summarizing the main findings and presenting possible directions for future work.

2 Background

This section provides the conceptual background necessary for understanding the subject of the thesis.

2.1 Iconclass

Iconclass is a classification system used to describe and organize iconographic subjects [20].³ It is structured as a hierarchical, tree-like system consisting of ten main categories, each represented by a digit from 0 to 9 (see Table 1). These categories are further divided into multiple levels of subcategories, forming branches that move from broader concepts to more specific subjects.

Each category in this hierarchy is assigned an alphanumeric notation along with its category name that serves as its identifier. The notations are formed by extending the code of a broader category with additional numbers or letters. In this way, the notation indicates the position of a category within the hierarchical structure. Its length and composition therefore reflect the level of specificity of the corresponding category.

This hierarchical principle is illustrated in Table 2 through an example. The notation 4 refers to the main category “Society, Civilization, Culture”. Its subcategory 46 denotes

²See <https://github.com/beriltezel/Evaluation-of-Text-Representation-Methods-for-Iconclass-Label-R>

³Unless indicated otherwise, this section draws on the Iconclass online edition, <https://iconclass.org>, both for the description of the system and for the notations and labels used as examples.

⁴Source: <https://iconclass.org> (Accessed September 1, 2026).

Table 1: The ten main categories of the Iconclass classification system⁴

Notation	Label
0	Abstract, Non-representational Art
1	Religion and Magic
2	Nature
3	Human Being, Man in General
4	Society, Civilization, Culture
5	Abstract Ideas and Concepts
6	History
7	Bible
8	Literature
9	Classical Mythology and Ancient History

“social and economic life, transport and communication”, while **46C** refers more specifically to “traffic and transport”. The notation **46C2** further narrows this category to “traffic on the water”, and **46C27** to “crew and passengers ~ ship”. At a deeper level, **46C273** denotes “celebration on board”, and **46C2731** specifies “celebration on board while crossing the equator (Neptune)”.

Table 2: Example of increasing specificity in the Iconclass hierarchy⁵

Notation	Label
4	Society, Civilization, Culture
46	social and economic life, transport and communication
46C	traffic and transport
46C2	traffic on the water
46C27	crew and passengers ~ ship
46C273	celebration on board
46C2731	celebration on board while crossing the equator (Neptune)

Alongside this hierarchical subdivision, Iconclass provides four features that allow the meaning of a notation to be stated more precisely: bracketed text, keys, doubling of the letter, and structural digits. None of these features is effective for the whole system. Each is offered only at the points where the corresponding distinction is intended, and the meaning it carries holds only within that part of the system.

The notation **25F23** is labeled “beasts of prey, predatory animals”, **25F23(...)** the same concept with the option to add a name, “beasts of prey, predatory animals (with NAME)”, and **25F23(LION)** is labeled “beasts of prey, predatory animals: lion”. This is how brack-

⁵Source: <https://iconclass.org> (Accessed September 1, 2026).

eted text works: at the points where a notation allows such an addition, a name is placed in parentheses at the end of the notation. It is an exception to descending in the hierarchy by extending a notation with digits, since it creates an alphabetical listing instead of a further subdivision. The notation with the empty bracket counts as a notation on its own and occupies a position in the hierarchy.

The notation 49M32 carries the label “book” and 49MM32 is labeled “book - MM - book open”. Doubling the upper case letter of a notation modifies its meaning in this manner. Within the part of the system where the doubling is offered, it has a specific meaning of its own, as in 25F for “animals” and 25FF for “fabulous animals (sometimes wrongly called ‘grotesques’); ‘Mostri’ (Ripa)”.

In 92B1217 the digit 2 following 92B1, the notation for “(story of) Jupiter (Zeus)”, denotes a love-affair. Digits used in this way are structural digits. Within a set of related notations, they carry a meaning of their own, so that the same digit denotes the same kind of subject for every character of that set.

Keys, finally, allow a further aspect to be added to a notation, and they play a decisive role in this thesis, since they take part in filtering the notations that enter the data set prepared in Section 4.1.2.

Keys are lists of concepts that are valid for a group of notations rather than for a single one. A key is written at the end of a notation, preceded by a plus sign and placed in brackets, and adds a shade of meaning to the notation it is attached to. The label of the resulting notation usually combines the two, the wording of the base notation followed by the wording of the key in parentheses, as illustrated in Table 3. Such a label is therefore in most cases longer than the label of the base notation alone.

Table 3: Example of a notation extended by a key⁶

Notation	Label
25F23(LION)	beasts of prey, predatory animals: lion
25F23(LION)(+1)	beasts of prey, predatory animals: lion (+ animals used symbolically)
25F23(LION)(+12)	beasts of prey, predatory animals: lion (+ heraldic animals)

The notation 25F23(LION)(+12) lies below 25F23(LION)(+1), which in turn lies below 25F23(LION). Keys are also subdivided, so that a key with more digits denotes a more specific aspect than a shorter one. A notation extended by a key is placed below its base notation in the hierarchy, and below the shorter keys of the same branch, if there are any. The digits of a key thus contribute to the position of a notation in the same way as the digits of the notation.

The labels of Iconclass are available in several languages, among them English, German,

⁶Source: <https://iconclass.org> (Accessed September 1, 2026.)

French, Italian, Portuguese and Japanese. Only the English labels are considered in this thesis.

2.2 The Nature of Retrieval in Iconclass

In the context of Iconclass, search can be understood as the process of matching a textual input to relevant Iconclass labels. Since each notation is connected to a textual label, the search process examined in this thesis is based on the comparison between the input query and the textual representations of Iconclass categories.

There are different ways to conduct a search in textual data. Two common approaches are detecting a string match between the input query and the Iconclass labels or calculating semantic similarity between them. A string match is based on whether the searched word or phrase occurs in one or more Iconclass labels, either as a complete string or as part of a longer text.

Calculating semantic similarity is a more complex task. This method uses embeddings to represent textual units in a vector space. An embedding can be defined as a representation of a word or phrase as a real-valued vector (see Section 3.1.1). These representations are constructed in such a way that texts that are closer in vector space are assumed to be more similar in meaning. As a result, similarity between textual units can be computed using vector similarity measures such as cosine similarity.

It is also important to note that one image can be classified under more than one notation, since there can be more than one meaning or concept extracted from that image. The assignment of the images to the labels is made by human catalogers. While the manual assignment of images to notations may seem conceptually similar to the search process in Iconclass, it is a different process and is not examined in this study.

2.3 Ground Truth

Ground truth is generally considered to be the data that is treated as correct or accurate for the purpose of evaluation in the machine learning community [8]. It functions as a reference point: the output of a method is compared against ground truth in order to judge how well the method performs. However, this does not mean that ground truth exists as an objective truth independent of interpretation. Muller et al. [18] describe ground-truth labeling as a process in which labels are defined, applied, revised, and checked through human work. Their point is that ground truth is not simply found in the data. It is produced through decisions about what should be treated as the reference label in a specific context.

For the evaluation of this thesis, ground truth refers to the set of Iconclass notations and their labels used as the reference results for a query. Defining such a reference standard is difficult because both language and semantic relevance are ambiguous. A short query may have more than one possible meaning. For example, a bank may refer to a financial institution or to a bench. Even when the intended meaning is clear, relevance is not always simply right or wrong. One label may describe a broad concept, while another may describe a more specific version of that concept. A label may also be connected to the query through a different kind of relation, such as an object, a place, a person, or a narrative scene. Therefore, several labels can be relevant at the same time, but not necessarily in the same way. This also makes ranking important. If only a limited number of returned results is evaluated, a relevant label must appear within that range in order to be recognized by the evaluation.

The structure of Iconclass adds another difficulty. Iconclass is hierarchical, and notations become more specific as one moves deeper into the system. Similar concepts may appear close to each other, but they may also appear in different branches because they belong to different subject areas. Creating ground truth therefore requires extensive knowledge of the Iconclass hierarchy, its vocabulary, and the different paths through which a concept may be reached. Without this knowledge, relevant notations may be missed. Still, such knowledge cannot remove all uncertainty. Meaning is not fixed in the same way as a formal rule; it depends on language, context, and interpretation. For this reason, ground truth is used here as a necessary evaluation standard, not as a final or absolute truth. Although the selected notations cannot remove all semantic uncertainty, they allow the different search methods to be evaluated under the same conditions.

Treating ground truth as an evaluation standard requires one further step that the preceding considerations do not provide on their own. Relevance has been described as a matter of degree, whereas the evaluation carried out in this thesis admits no degrees: a notation either belongs to the reference set of a query or it does not, and a notation outside that set counts as incorrect. This boundary follows from the requirements of measurement rather than from the material, and its uniform application across all methods is what makes the comparison possible.

3 Theoretical Foundations

This chapter introduces the theoretical foundations of the search methods used in this thesis. Since the task consists of comparing textual queries with Iconclass labels, the focus lies on methods for representing text and for measuring similarity. The chapter first introduces vector embeddings, the way a text is encoded from tokens to a single vector, and cosine similarity as the measure used to compare such representations. Following

are the retrieval methods used in this thesis: BM25 as a lexical model, SBERT and EmbeddingGemma as text embedding models, and finally, SigLIP 2 as a vision-language model whose text encoder is used as an additional text representation method.

3.1 Representing and Comparing Texts

3.1.1 Vector Embeddings

Vector embeddings are numerical representations of text in a high-dimensional vector space [13]. In the context of information retrieval, they are used to represent queries and documents in a way that allows semantic comparison [12; 17].

The central idea behind embeddings is that texts with similar meanings should receive similar vector representations [17]. This enables two texts to be matched even when they express the same concept using different vocabulary, which is not possible for lexical models relying on exact term matches [13; 16; 24].

An embedding can be represented as a vector:

$$\mathbf{x} = (x_1, x_2, x_3, \dots, x_n) \quad (3.1)$$

where each component corresponds to one dimension in the vector space. In practice, embedding models use high-dimensional vectors to capture semantic information [17; 21].

This representation allows both queries and documents to be mapped into the same vector space. Documents that are semantically similar to a query are expected to be located closer to it, while less relevant documents are positioned further away [12; 13].

As a result, embeddings form the basis of semantic search systems, where ranking is performed based on the similarity between query and document representations [23; 24].

3.1.2 Encoding a Text as a Single Vector

A text is not processed as a whole from the very beginning. An input text is first split into smaller units, called tokens, which correspond to words or to parts of words [21]. What follows from this step differs between the two kinds of methods. Lexical methods split a text at whitespace, so that every token is a word as it is written, and the comparison is then carried out on these words directly [13], as described in Section 3.2.1.

Embedding models split a text according to a fixed vocabulary of units. This vocabulary is derived from a large text corpus before the model is trained and remains unchanged afterwards [9; 28]. A frequent word forms a single token, while a less frequent word is

divided into several parts of a word [28], and the model produces one representation per token [3].

Since retrieval requires a single vector per text, these token representations then have to be combined into one. The most common way to do this is to average them [15], which gives every token of the input the same weight. Another strategy is to not take all the vectors into account but to read the representation from a single position of the sequence [3]. Alternatively, the combination can also be learned, so that the model decides how much each token contributes to the final vector [11]. The embedding methods compared in this thesis use the first two options and this choice matters because it determines whether every token of a label contributes to its representation.

For the transformer-based models, the number of tokens that can be processed in a single input is limited [33]. This applies to all embedding-based methods considered in this thesis. Longer inputs are cut off at this limit, so that the part of a text beyond it does not contribute to its vector representation.

3.1.3 Cosine Similarity

To use vector embeddings for retrieval, a similarity measure is required to compare query and document vectors. One of the most commonly used measures in this context is cosine similarity [13; 31]. Cosine similarity compares two vectors based on the angle between them rather than their magnitude. This is particularly useful for text embeddings, where the direction of a vector encodes semantic information [16; 17].

Given two vectors A and B , cosine similarity is defined as:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} \quad (3.2)$$

where $A \cdot B$ denotes the dot product and $\|A\|$, $\|B\|$ are the magnitudes of the vectors. The resulting value ranges from -1 to 1 , where higher values indicate greater similarity. In retrieval applications with text embeddings, values typically lie between 0 and 1 [13; 17].

In a search setting, cosine similarity is used to compare the embedding of a query with the embeddings of documents. The resulting similarity score is then used to rank documents, where higher scores indicate higher relevance [17; 32].

Compared to lexical scoring functions such as BM25, cosine similarity operates on semantic representations. This allows relevant documents to be retrieved even when they do not share exact query terms [16; 22; 23; 24].

3.2 Text Representation Methods

3.2.1 Best Matching 25

BM25 (Best Matching 25) is a ranking function used in information retrieval to estimate how relevant a document is for a given query. It is based on the probabilistic relevance framework and is one of the foundational lexical retrieval models [25]. BM25 scores a document over its terms. The tokens of a text are normalized, for example by lowercasing them. The resulting units are the terms under which a document is indexed and retrieved [13]. Only terms that occur in both the query and the document contribute to the score, so a document that describes the same concept in different words receives no score at all.

A commonly used formulation of the BM25 score of a document d with respect to a query q is:

$$\text{score}(d, q) = \sum_{t \in q} \text{idf}(t) \cdot \frac{\text{tf}(t, d)(k_1 + 1)}{\text{tf}(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (3.3)$$

Every matching term contributes to the score according to three quantities. The first is the term frequency $\text{tf}(t, d)$, the number of times t occurs in d [13]. It does not enter the score linearly, but through a saturation function, meaning that the first occurrence of a term has a stronger impact than each following one [25]. The second is the inverse document frequency $\text{idf}(t)$, which weights a term by how rare it is in the collection. Terms that occur in few documents are more useful for telling documents apart, and therefore matter more [26; 30]. The third is the ratio of the document length $|d|$ to the average document length avgdl [25]. Longer documents contain more terms, which could lead to higher scores without being more relevant, and this factor compensates for that [25].

Two parameters control how strongly two of these effects apply. The parameter k_1 determines how quickly the term-frequency contribution saturates. The parameter b , which takes values between 0 and 1, determines how much document length is taken into account: at $b = 0$ it is ignored, and at $b = 1$, the normalization is applied in full [25].

The inverse document frequency can be formulated in different ways. In the probabilistic relevance framework, the weight of a term is derived from the number of documents that contain it and the number that do not. This yields the form used in this thesis [25]:

$$\text{idf}(t) = \log \left(\frac{N - \text{df}(t) + 0.5}{\text{df}(t) + 0.5} + 1 \right) \quad (3.4)$$

Here N is the number of documents in the collection and $\text{df}(t)$ the number of documents containing t . The added constants of 0.5 keep the expression defined when a term occurs in none or in all of the documents.

3.2.2 SBERT

Sentence-BERT (SBERT) is a language model that is based on a transformer-based language model BERT (Bidirectional Encoder Representations from Transformers), which was trained on large amounts of unlabeled text.

BERT represents each token of an input sequence as a vector depending on its context [3]. Comparing two texts with BERT means giving both of them to the network as a single input, which returns a score for that particular pair. No separate representation of either text is produced, so every comparison requires a new pass through the network [24], which makes large-scale ranking computationally very expensive. SBERT addresses this by encoding each text separately. Two texts are passed through the same network with shared weights, a structure referred to as a siamese network, and the training signal is applied to the resulting vectors rather than to a joint representation. Semantically similar texts are mapped to nearby vectors, while dissimilar texts are placed further apart [24].

Encoding a text therefore proceeds in two steps. The transformer first produces one vector for each of the inputs. These are then reduced to a single vector by pooling, in the standard case by averaging them [24]. The result is a representation of fixed size, regardless of the length of the input. However, the input is not unlimited. Beyond a certain number of tokens a text is cut off, and where this limit lies depends on the model.

The model was designed so that texts with similar meanings receive vectors pointing in similar directions [23]. Cosine similarity is therefore an appropriate measure when a search is performed, and the ranking described in Section 3.1.3 can be applied to these representations directly. Because each text is encoded independently, the representations of a collection can be computed once and stored, so that a search only requires encoding the new text and comparing it with the stored vectors [24].

3.2.3 EmbeddingGemma

EmbeddingGemma is a lightweight text embedding model based on the Gemma 3, a generative language model [4]. However, the working principle of EmbeddingGemma is different from generative language models. A generative model reads a text from left to right, since it is trained to predict how a sequence continues. For EmbeddingGemma the attention mechanism operates in both directions. Every token is represented in the context of the complete input rather than of the preceding part alone [35]. The resulting token vectors are averaged and passed through two linear projections, which map the pooled vector into the dimensionality of the final representation [35].

This representation has 768 dimensions and the model produces it in a way that its leading dimensions already form a usable representation on their own, while the later

ones add increasingly fine distinctions [10; 35]. A shorter vector can therefore be obtained by keeping only the first dimensions and scaling the result to unit length again, so that it can be compared in the same way as a full-length one [10; 35]. Reducing the number of dimensions lowers storage requirements and comparison time, and how far this is done depends on the application (see Section 4).

The input has to be provided in a fixed form. EmbeddingGemma expects a short instruction in front of the text, which states the purpose the representation is intended for. In retrieval, a text used as a search query is encoded as “task: search result | query: {content}”, and a text belonging to the collection being searched as: “title: {title | “none”} | text: {content}” [4; 35]. The same text therefore receives different representations depending on which instruction precedes it, so a representation depends not only on the text itself but also on the purpose it is intended for. Together with the instruction, the model accepts inputs of up to 2048 tokens [4].

A search is therefore carried out on two differently marked inputs. Both are mapped into the same vector space, in which the vectors tend to be distributed across the space rather than concentrated in a small region [35]. As a result, texts of similar meaning are still assigned distinguishable positions instead of nearly identical ones. Since the model was trained so that the similarity of two texts is reflected in the angle between their vectors [35], the cosine similarity is an appropriate similarity method to compare them.

3.2.4 SigLIP 2

SigLIP (Sigmoid Loss for Language-Image Pre-Training) is a vision-language model that learns representations for images and texts in a common embedding space [38]. It consists of two encoders: an image encoder that maps images to vector representations and a text encoder that maps textual input to vector representations. During training, every image embedding of a batch is combined with every text embedding and the model learns to classify each combination as a matching or non-matching pair. This classification is formulated as a sigmoid loss over the individual pairs, allowing each pair to be considered independently [38].

SigLIP 2 is a later version of SigLIP and extends the original approach as a family of multilingual vision-language encoders. It uses the same architecture as SigLIP, but the text encoder is no longer restricted to English, since a multilingual tokenizer is used [34]. Since the two encoders are separate modules, each of them can be used on its own once training is complete [7].

Since this thesis focuses only on comparing textual data, only the text encoder is used to represent Iconclass labels and search queries, without involving image representations.

This difference in training provides the motivation for RQ1.2 (see Section 1.1). Since the SigLIP 2text encoder learns textual representations under a different objective, it can be expected to behave differently from the text-only models. SBERT and EmbeddingGemma were optimized on pairs of texts [24; 35], so distances in their embedding spaces reflect how closely two texts are related in meaning. The training objective of the SigLIP 2text encoder, by contrast, is defined over image-text pairs rather than pairs of texts [34]. The encoder learned to place the vector representation of the text near the vector representation of the image it describes, and similarity between two texts follows from this only indirectly. The two texts end up close to each other mainly if they would describe similar pictures. For a vocabulary of depictable subjects such as Iconclass, this can work in either direction. A query such as “man carrying a sheep on his shoulders” and a label such as “the Good Shepherd” share no words, yet they describe the same picture, and a representation shaped by captions may connect them where a lexical method cannot. In the other direction, Iconclass separates subjects that are difficult to tell apart as pictures. Subjects such as “David with the head of Goliath”, “Judith with the head of Holofernes” and “Perseus with the head of Medusa” all show a figure holding a severed head and belong to different branches of the hierarchy, whereas a model trained on image-text relationships rather than relationships between textual inputs has little reason to keep them apart.

A further difference between SigLIP 2 and the text-only models lies in how a single text vector is obtained from the token representations. The text encoder was trained with a fixed length of 64 tokens, so every input is padded to this length and no text longer than the limit can be encoded [7; 34]. The representation is then taken from the final position of the sequence and passed through a linear projection [6; 7]. The padding therefore fixes the position from which the representation is read, independently of the length of the text itself, and the content of a text has to be gathered at that position while the text is being encoded. SBERT and EmbeddingGemma, in contrast, average the token vectors to obtain the text vector [4; 24]. A projection is applied in EmbeddingGemma as well [35], so the difference does not lie in this step but in the absence of averaging over the token vectors.

Retrieval then proceeds as for the text-only models. Every label is encoded once and stored, and a query is compared with the stored representations by cosine similarity.

4 Methodology

This chapter describes the methodological approach used to compare different retrieval models on Iconclass data. The aim of the implementation was to retrieve relevant Iconclass categories for a given textual query and to compare how lexical and semantic retrieval methods perform in this setting.

4.1 Preparation of the Data

4.1.1 Extraction of the Iconclass Data

The data preparation was based on the Iconclass data available through the Python Iconclass package [20]. The source data contains 40,672 notations, all of which were included in the pool of notations considered in this thesis. The source data further assigns to each notation the keys that may be applied to it. Combining every notation with each of its keys produces 1,330,123 notations of the form `notation(+key)`. Together with the notations read before, this results in a pool of 1,370,795.

For each notation of this pool, the corresponding Iconclass node was accessed and the English label was extracted. Notations without an English label were excluded, since they cannot be used for a text-based comparison with the queries. This applied to five notations.

4.1.2 Construction of the Search Collection

Keys are defined for groups of notations rather than for individual notations, so not every combination formed in this way corresponds to a concept that is used. At more than 1.3 million notations, the pool is also more than thirty times as large as the set of notations contained in the source data.

The pool was therefore filtered and reduced to the notations that occur in practice. Notations without a key were kept. A notation with a key was kept only if it was listed in the file that has all the notations that users have searched for in Iconclass. If this was not the case, the notation was skipped. This removed 1,315,124 notations and resulted in a candidate set of 55,666 notations, 14,999 of which contain a key.

Restricting the candidate set in this way keeps the recommendations within the part of the system that has been used to search for concepts. This ensures that the models were compared using identical Iconclass data.

4.1.3 Iconclass Database

After the Iconclass notations were filtered, the selected data was stored in a SQLite database. This database was created to keep the prepared Iconclass data in one structured format. In addition to the labels, the database holds the hierarchical relations between the notations, which are required for the hierarchy-based evaluation described in Section 4.4.6.

In the database, the data is stored in a single table named `iconclass`. Each row in this table represents one included Iconclass entry and holds its notation, the language of the

label, the label itself, the parent notation, and the depth.

The notation and its label form the main data used for retrieval. The notation identifies the Iconclass category, while the label is the text that is compared with the input query.

The parent and depth values store the position of each notation in the Iconclass hierarchy. During database creation, the path of each Iconclass node was read from the Iconclass package. The path of a top-level category consists only of the category itself, so its depth is one and its parent value remains empty, as the ten categories form the highest level of the hierarchy.⁷ Two labels belonging to different top-level categories consequently share no ancestor in the stored hierarchy.

For all remaining notations, the preceding node in the path was stored as the parent and the depth was set to the number of nodes on the path. Extensions form additional levels below the notation they belong to. For example, the parent of 11A11(+3) is 11A11 and the parent of 11A1(+31) is 11A1(+3).

4.2 Construction of Ground Truth

The ground truth used in this thesis consists of 52 queries of the kind that catalogers use when searching Iconclass for a suitable category, such as “reading a book”, “saint holding a monstrance”, or “portrait”. For each query, the ground truth lists the Iconclass notations that are relevant for it.

The queries were selected to cover the situations in which the wording of a query and the wording of an Iconclass label can differ. They include the same subject expressed in different words, as in “soldier on a horse” and “officer riding a horse” or “Jesus without a beard” and “Christ beardless”, as well as the same subject at different levels of detail, as in “lion” and “sleeping lion” or “portrait”, “portrait of a girl” and “portrait of a toddler”. Single terms such as “bridge” or “castle” are contrasted with descriptive phrases such as “god the father holding the body of christ” and concrete subjects with abstract concepts such as “hatred” or “jilted lover”. Named persons are represented by queries such as “Saint Agnes” and “Jupiter”. The misspelled variant “Jupyter” was assigned the same relevant notations as “Jupiter”, since a spelling error does not change the subject a user is looking for. In an ideal case, both queries therefore lead to the same recommendations, and the pair makes visible how far the methods still reach these notations when the query is spelled incorrectly.

The queries and notations considered relevant for them were gathered together in consul-

⁷The Iconclass Python package lists the ten top-level categories as children of an additional node, so that the data forms a single tree. This node does not correspond to an Iconclass category and is not part of the paths returned for the categories. It was therefore not stored in the database.

tation with Iconclass experts⁸. A notation counts as relevant if the concept expressed by the query is included in what its label describes, either as the main subject of the label or as an element of the scene it describes. Since a concept can be represented in several places of the classification system, one query can have more than one relevant notation and this number ranges from 3 to 269 across the 52 queries. The ground truth contains 1,615 distinct notations in total.

Some notations that could also be considered relevant were removed by the filtering rule described in Section 4.1.2 so the ground truth was limited to the notations contained in the search collection.

4.3 Implementation of the Methods

4.3.1 BM25

BM25 was applied directly to the labels of the search collection, where each label was treated as one document. The document length $|d|$ in Equation 3.3 is thus the number of terms of a label. In Equation 3.4, N is the number of labels of the search collection and $\text{df}(t)$ is the number of those labels that contain the term t . Lowercasing was the only normalization applied, both to the labels and to the queries of the ground truth.

The parameters were set to $k_1 = 1.5$ and $b = 0.75$. Without parameter optimization, $b = 0.75$ and a value of k_1 between 1.2 and 2 are reported as reasonable [13], and the same ranges are reported as effective across collections [25]. The value of k_1 was set to the middle of that range.

For each of the 52 queries, the BM25 score defined in Equation 3.3 was computed for every label of the search collection. Labels with a score of zero were left out, since they share no term with the query. The remaining labels were ranked by decreasing score and the 20 highest were kept. A query can therefore also return fewer than 20 labels if fewer than 20 labels contain any of its terms.

4.3.2 SBERT

The model used for SBERT is `all-MiniLM-L6-v2`, a compact six-layer model for English text with roughly 22 million parameters [36]. It represents a text as a vector with 384 dimensions and forms this vector by averaging the token representations [24; 29], as described in Section 3.1.2. Inputs longer than 256 tokens are truncated [29], which did not affect the labels of the search collection or the queries.

⁸Many thanks to Etienne Posthumus and Hans Brandhorst for their inputs.

The labels of the search collection were read from the database described in Section 4.1.3 and encoded with `all-MiniLM-L6-v2`. The labels and the queries were passed on unchanged, since the model applies its own normalization. This process includes lowercasing and any preceding change to the text would alter the resulting tokens. Each vector of the labels was scaled to unit length and stored together with its notation.

For each of the 52 queries, the query was encoded with the same model and compared with all stored vectors by cosine similarity (see Section 3.1.3). The 20 labels with the highest similarity were kept as the result for that query.

4.3.3 EmbeddingGemma

The model used for EmbeddingGemma is `embeddinggemma-300m` with 308 million parameters [35]. The vectors were stored with 512 of the 768 dimensions the model produces, using the shortening described in Section 3.2.3 [10].

The notations of the search collection were read from the Iconclass package and the labels were encoded. Each label was preceded by the instruction for texts of the collection and each query by the instruction for queries, both given in Section 3.2.3. The resulting vectors were stored in a database together with the notation of their label.

The 52 ground truth queries were encoded with the same model and each query was compared with all stored vectors by cosine similarity (see Section 3.1.3). The 20 notations with the highest similarity form the result for a query.

4.3.4 SigLIP 2

For the implementation of SigLIP 2, only the text encoder of the model `siglip2-so400m-patch16-naflex` was used [5; 34], since the comparison was carried out only between the ground truth queries and the Iconclass labels. The text encoder represents each text as a vector of 1152 dimensions [1; 7].

The labels of the search collection were read from the Iconclass package and passed to the processor of the model. The processor lowercases the text and pads it to a length of 64 tokens, the form the model was trained with [7; 34]. Inputs that exceed this length are truncated [7], which in this setting affected 83 labels of the search collection. The vectors of the labels were scaled to unit length and stored together with their notation in a database.

The 52 ground truth queries were then encoded in the same way, and each query was compared with all stored vectors by cosine similarity (see Section 3.1.3). The 20 notations with the highest similarity form the result for a query.

4.4 Evaluation Metrics

For a query q , let $P(q)$ represent the predicted and ranked list of notations a method returns and $G(q)$ the set of notations the ground truth defines as relevant. Throughout this section, $|\cdot|$ denotes the number of elements of a set. A returned notation contained in $G(q)$ is a true positive, a returned notation not contained in $G(q)$ is a false positive, and a notation of $G(q)$ that the method does not return is a false negative [13]. The definitions follow from $P(q)$ and $G(q)$:

$$TP(q) = |P(q) \cap G(q)|, \quad FP(q) = |P(q) \setminus G(q)|, \quad FN(q) = |G(q) \setminus P(q)|$$

A fourth case remains, the notations that are neither returned nor relevant. The true negatives do not enter any of the metrics used here, because nearly the entire search collection falls into this case for every query. The collection holds 55,666 notations (see Section 4.1.2), while a query has between 3 and 269 relevant ones, so a measure that counts true negatives would sit close to its maximum for any method. Accuracy demonstrates what follows from this. A method that returned nothing at all would still classify the overwhelming non-relevant majority correctly. Its accuracy score would come close to perfect. A measure defined over the non-relevant part of the collection therefore reports the size of that part rather than the quality of a result list [13]. The metrics below are therefore built only from true positives, false positives and false negatives.

The number of notations a method returns never exceeds 20 and varies between queries. SBERT, EmbeddingGemma and SigLIP 2 keep the 20 highest-ranked notations for every query. BM25 first discards labels that share no term with the query, so its list can be shorter (see Section 4.3.1). The metrics below therefore use the number of notations a method actually returns, not the constant 20. If a method returns nothing for a query, all metrics defined in this section take the value zero for that query.

4.4.1 Precision

Precision is the share of the returned notations that are relevant [13]. For a query q , it is defined as

$$\text{Precision}(q) = \frac{TP(q)}{TP(q) + FP(q)} = \frac{|P(q) \cap G(q)|}{|P(q)|}. \quad (4.1)$$

The denominator is the number of notations the method returns for q . Every method is therefore measured on the list it produces. A method that returns five notations of which two are relevant reaches the same precision as a method that returns twenty of which eight

are relevant. A value is computed for each query, and the figure reported for a method is the mean over the 52 queries. Precision describes the returned list alone. It says nothing about the relevant notations that a method leaves out. A list of twenty relevant notations reaches a precision of 1 whether the ground truth holds twenty relevant notations or two hundred.

4.4.2 Recall

Recall is the share of the relevant notations that a method returns [13]. For a query q , it is defined as

$$\text{Recall}(q) = \frac{TP(q)}{TP(q) + FN(q)} = \frac{|P(q) \cap G(q)|}{|G(q)|}. \quad (4.2)$$

The denominator is the size of the ground truth of q and does not depend on the method. Recall therefore measures the returned results against the whole ground truth, not against the number of notations a method returns.

A method returns at most 20 notations, while the ground truth of a query holds between 3 and 269 notations (see Section 4.2). For 28 of the 52 queries, the ground truth holds more than 20 notations. Since a relevant label is only recognized by the evaluation when it appears among the returned results (see Section 2.3), the highest recall reachable for such a query is $20/|G(q)|$ instead of 1. The limit is lowest for “landscape”, the query with the largest ground truth, where it falls to 0.074.

The limit differs from query to query. Recall values are therefore not comparable between queries. A comparison between the four methods stays valid, because all of them return at most 20 notations and are measured against the same ground truth.

4.4.3 F_1 -Score

Precision and recall describe opposite sides of a result list, and a method can raise one while lowering the other. The F_1 -score combines the two into a single value. It is their harmonic mean [13]. For a query q , it is defined as

$$F_1(q) = 2 \cdot \frac{\text{Precision}(q) \cdot \text{Recall}(q)}{\text{Precision}(q) + \text{Recall}(q)}. \quad (4.3)$$

The harmonic mean stays low as long as one of the two values is low, whereas an arithmetic mean would not. A high F_1 -score is therefore out of reach for a method that favors one of the two, since a weak precision or weak recall pulls the combined value down [13]. If precision and recall are both zero, the F_1 -score is set to zero.

Recall enters the equation, so the F_1 -score inherits its limit. For the 28 queries whose ground truth holds more than 20 notations, the highest reachable F_1 -score also lies below 1.

4.4.4 R-Precision

R-Precision judges as many returned notations as there are relevant ones for a query. Let $R = |G(q)|$ and let $P_R(q)$ denote the first R notations of $P(q)$. Following [13], R-Precision is the precision measured at position R , which gives the equation

$$\text{R-Precision}(q) = \frac{|P_R(q) \cap G(q)|}{|G(q)|}. \quad (4.4)$$

The number of judged positions thus follows the size of the ground truth. For example, a query with three relevant notations is judged on three returned notations, and a query with fifteen relevant notations on fifteen positions. Measured at position R , precision and recall take the same value. Both denominators equal R , so a single number describes both sides of the result list.

For 28 of the 52 queries, the ground truth holds more than 20 notations. For these queries, R exceeds the number of notations a method returns, so $P_R(q)$ contains every returned notation and nothing is truncated. R-Precision then takes the same value as recall and carries the same limit. The adaptation of the judged positions therefore takes effect only for the remaining 24 queries, whose ground truth holds at most 20 notations.

R-Precision depends only on which notations appear up to position R , not on their order within that range.

4.4.5 Mean Average Precision (MAP)

Mean average precision builds on a value computed for a single query, so average precision is defined as a foundation. Average precision reads the returned results from the highest-ranked notation downwards. Each time a relevant notation is reached, the precision over the notations up to and including that position is measured. A relevant notation reached early contributes a higher precision value to the sum, since that value is computed over fewer notations. The same notation reached later contributes a lower one. The measured values are added and then divided by the size of the ground truth.

Let $\text{Precision}(q, k)$ be the precision over the first k notations of $P(q)$, and let $\text{pos}(p)$ be the position of a notation p in $P(q)$. Average precision is then defined as

$$\text{AP}(q) = \frac{1}{|G(q)|} \sum_{p \in P(q) \cap G(q)} \text{Precision}(q, \text{pos}(p)). \quad (4.5)$$

Average precision therefore rises when a method places relevant notations at the top of its list.

The denominator is the size of the ground truth, so average precision carries the same limit as recall. For the 28 queries whose ground truth holds more than 20 notations, the highest reachable value lies below 1.

Mean average precision is formed for each method separately. It is the arithmetic mean of the average precision values a method reaches over the 52 queries [13], which gives

$$\text{MAP} = \frac{1}{52} \sum_q \text{AP}(q), \quad (4.6)$$

where the sum runs over the 52 queries. Each query contributes one value to this mean, so a query with 269 relevant notations affects the result as much as a query with three [13].

4.4.6 Wu-Palmer Similarity

Wu-Palmer similarity derives the similarity of two concepts from their positions in a hierarchical classification and expresses it as a value between zero and one [37]. For a relevant notation g and a retrieved notation p , the measure is

$$\text{WUP}(g, p) = \frac{2 \cdot \text{depth}(\text{lca}(g, p))}{\text{depth}(g) + \text{depth}(p)} \quad (4.7)$$

where $\text{lca}(g, p)$ is the deepest notation that is an ancestor of both g and p , and depth is the depth value stored for a notation (see Section 4.1.3). The numerator grows with the depth of the shared ancestor. Two notations that follow a common path for longer therefore reach a higher value. The denominator grows with the depths of the two notations, which keeps the values comparable across the levels of hierarchy. Both notations enter the denominator in the same way, so the measure is symmetric and the order of the two notations does not affect the result.

Wu and Palmer define the measure through the number of nodes on three paths [37]. One path runs from the shared ancestor to the root, and two run from g and from p up to the shared ancestor. The stored depth values give all three numbers without following the paths. The first is the depth of the shared ancestor. The other two are the depths of g and p , each reduced by that value. Substituting the three numbers into the definition of Wu and Palmer gives Equation 4.7 [14].

Two particular cases considered in this thesis take fixed values. Two identical notations receive 1.0, which is the maximum value of the measure. Two notations without a shared ancestor receive 0.0. This applies to every pair from different main categories, since the ten main categories form the highest level and have no common parent (see Section 4.1.3).

Equation 4.7 compares two single notations, while a query has a set of relevant notations defined in the ground truth and a list of retrieved notations. Neither the sizes of the two sets nor the order of their elements determine which relevant notation is compared with which returned one. The measure is therefore applied to every pair, and the resulting values are reduced to one value per query.

The reduction decides what the value rewards. An average over all pairs weights a pair without a shared ancestor like a pair that matches, so its value follows the sizes of the two sets [19]. The maximum over all pairs rests on one pair and ignores every other notation [19]. The reduction used here keeps for each notation only the highest value it reaches against the other set, and averages these values [19; 27]. Every notation then contributes exactly one of these values to the average, independent of the number of notations the other set has.

Averaging over the relevant notations gives the first directions,

$$\text{WUP}_{G \rightarrow P}(q) = \frac{1}{|G(q)|} \sum_{g \in G(q)} \max_{p \in P(q)} \text{WUP}(g, p). \quad (4.8)$$

Each relevant notation is compared with the whole returned list and keeps its closest counterpart. Equation 4.8 therefore states how completely the relevant notations of a query are approached. A relevant notation that is returned exactly contributes 1.0. One that is approached from a neighboring branch contributes a fraction. One whose main category is missing from the returned list contributes 0.0.

Averaging over the returned notations gives the second direction,

$$\text{WUP}_{P \rightarrow G}(q) = \frac{1}{|P(q)|} \sum_{p \in P(q)} \max_{g \in G(q)} \text{WUP}(p, g). \quad (4.9)$$

Each returned notation is compared with the whole ground truth. Equation 4.9 therefore states how close a method stays to the relevant notations. The two directions divide by the size of the ground truth and by the number of returned notations, which are the denominators of recall and of precision.

The two directions can differ for the same query. A method that keeps its notations inside one narrow branch of the hierarchy reaches a high value in Equation 4.9, since each returned notation stands near a relevant one. Equation 4.8 stays low, since the relevant notations outside that branch are approached by nothing. Reporting the larger of the two values would leave the missing coverage unpenalized [27]. Their arithmetic mean is

$$\text{WUP}_{\text{mean}}(q) = \frac{\text{WUP}_{G \rightarrow P}(q) + \text{WUP}_{P \rightarrow G}(q)}{2}. \quad (4.10)$$

The mean stays low as long as one direction is low [27]. A high mean therefore requires returned notations that stay close to the ground truth and cover it at the same time. All three values are reported for each method as the mean over the 52 queries, since the mean of the two directions alone does not show whether they agree.

Selecting the closest match is not a one-to-one assignment. One notation can be the closest match of several notations of the other set, and others of none. The contribution of a notation therefore does not depend on the remaining pairs. The two directions also average over a different number of values, at most 20 in Equation 4.9 and between 3 and 269 in Equation 4.8. A single notation weighs more in the second direction than in the first. The rank of a notation within the returned list enters none of the three values.

The three values are computed on the returned notations and the ground truth as they stand. A notation that is an ancestor of another notation in the same set is removed. Such notations are excluded when the entries of a set are inherited along the hierarchy [19]. The relevance of a notation is decided from its own label and is not inherited along the hierarchy (see Section 4.2). So a notation and its ancestor are judged independently, and both stay in the set.

5 Results

This section reports the outcome of the evaluation described in Section 4. BM25, SBERT, EmbeddingGemma, and the text encoder of SigLIP 2 were applied to the same set of 52 queries and to the same collection of 55,666 Iconclass notations. The ground truth was constructed manually for these queries, as described in Section 4.2. It contains 2,015 query-notation assignments across 1,615 distinct notations. The number of relevant notations per query varies considerably. The metrics used for the evaluation are defined in Section 4.4. Unless a subset of the queries or a single query is named explicitly, every metric value represents the arithmetic mean over the 52 queries, rounded to four decimal places.

Table 4 presents the results of each method for the eight metrics used in the evaluation. No method holds the highest value on all eight metrics. The precision values of BM25, SBERT and EmbeddingGemma differ no more than 0.0077, with 0.3239, 0.3288 and 0.3212. On the seven remaining metrics, SBERT and EmbeddingGemma perform better than BM25. EmbeddingGemma attains the highest value on recall, the F_1 -score, R-Precision, MAP, and Wu-Palmer similarity in the $G \rightarrow P$ direction. SBERT attains the highest value on precision and on Wu-Palmer similarity in the $P \rightarrow G$ direction. The difference between

Table 4: Mean scores of the four methods over the 52 queries

Method	Precision	Recall	F_1 -Score	R-Prec.	MAP	Wu-Palmer		
						$G \rightarrow P$	$P \rightarrow G$	mean
BM25	0.3239	0.2507	0.2208	0.2082	0.1737	0.5771	0.6140	0.5955
SBERT	0.3288	0.3039	0.2490	0.2455	0.2155	0.6123	0.7290	0.6707
EmbeddingGemma	0.3212	0.3272	0.2538	0.2637	0.2218	0.6228	0.7215	0.6721
SigLIP 2	0.1788	0.2041	0.1420	0.1466	0.1301	0.5753	0.4744	0.5248

the two methods on the Wu-Palmer mean is 0.0015. SigLIP 2 records the lowest value of the four methods on all eight metrics. Its largest difference to the other three methods occurs in the $P \rightarrow G$ direction of Wu-Palmer similarity, where it reaches 0.4744 against 0.7290 for SBERT. Figure 1 shows the same values.

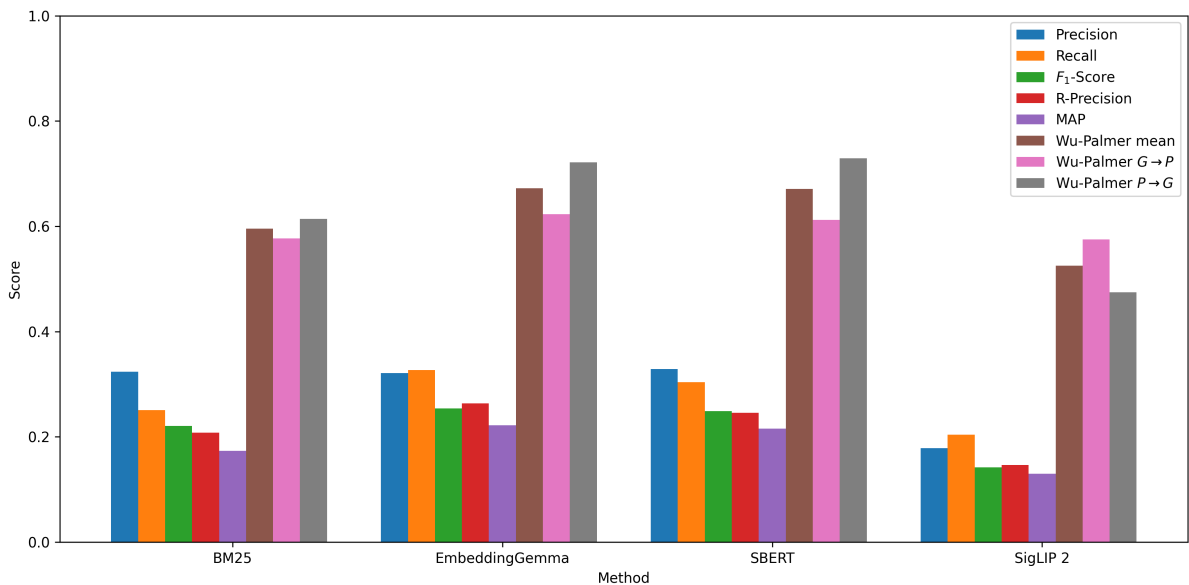


Figure 1: Mean scores of the four methods over the 52 queries

Recall indicates which method returns the notations that the ground truth defines as relevant. EmbeddingGemma returns the largest share of them with 0.3272, followed by SBERT with 0.3039, BM25 with 0.2507 and SigLIP 2 with 0.2041. Precision does not reproduce this order. The three methods with the highest precision values differ by 0.0077, whereas the difference between them on recall is 0.0765. BM25 ranks the third on recall and the first on precision, while EmbeddingGemma ranks first on recall and third on precision. The F_1 -score, follows the order of recall with 0.2538 for EmbeddingGemma, 0.2490 for SBERT, 0.2208 for BM25 and 0.1420 for SigLIP 2. Recall reaches 1.0000 in six of the 208 query-method pairs. These are SBERT on “making lace” and “sleeping lion”, EmbeddingGemma on “fireman”, and SigLIP 2 on “making lace”, “elderly woman” and “saint holding a monstrance”. The ground truth of each of these six queries contains at

most eight notations. BM25 does not reach 1.0000 on any query, and its highest recall is 0.9167 on “castle”.

One aspect has to be added for the precision of BM25, since the number of notations it returns varies from query to query, as described in Section 4.3.1. Table 5 lists the seven queries for which BM25 returned fewer than 20 notations. Three of these result lists are empty. Precision, recall and average precision therefore take the value zero for “fireman”, “chandelier” and “Jupyter” because the method returns nothing for them, and not because of the ranking. The shorter lists also give the two highest precision values of BM25: 1.0000 on “hatred” and 0.8571 on “hill”. SBERT, EmbeddingGemma and SigLIP 2 return 20 notations for every query, since cosine similarity gives a value for every notation of the collection.

Table 5: Queries for which BM25 returned fewer than 20 notations

Query	Returned	Relevant returned	Precision
“fireman”	0	0	0.0000
“chandelier”	0	0	0.0000
“Jupyter”	0	0	0.0000
“hatred”	2	2	1.0000
“commanding gesture”	9	0	0.0000
“castle”	14	11	0.7857
“hill”	14	12	0.8571

The queries of the ground truth range from single terms to descriptive phrases, as described in Section 4.2. Table 6 reports precision and recall by the number of words a query contains, where n denotes the number of queries in a group. On the 19 queries consisting of one word, BM25 attains the highest precision of the four methods with 0.5207, while SBERT reaches 0.4079 and EmbeddingGemma 0.4053. On the 25 queries consisting of 2 – 3 words, this order changes. BM25 reaches 0.2420, SBERT 0.3320 and EmbeddingGemma 0.3100. SigLIP 2 is the only method whose precision increases between these two groups, from 0.1658 to 0.2160. On the 8 queries of four words or more, BM25 falls behind SBERT and EmbeddingGemma again with 0.1125, against 0.1313 for SBERT and 0.1562 for EmbeddingGemma. Recall does not follow this pattern. BM25 remains below SBERT and EmbeddingGemma on the two groups with the shorter queries, but exceeds SBERT on the queries of four words or more with 0.2801 against 0.1704. Between the group of one worded queries and the group of queries with four words or more, the precision of BM25 decreases by 0.4082. The corresponding decrease amounts to 0.2766 for SBERT, 0.2490 for EmbeddingGemma and 0.0720 for SigLIP 2.

The size of the ground truth affects the values as well. For 28 of the 52 queries, the ground truth contains more than 20 notations. As a result, the highest reachable recall

Table 6: Mean precision and recall by the number of words of a query

Method	1 Word (n = 19)		2 - 3 Words (n = 25)		≥ 4 Words (n = 8)	
	Precision	Recall	Precision	Recall	Precision	Recall
BM25	0.5207	0.2018	0.2420	0.2785	0.1125	0.2801
SBERT	0.4079	0.2306	0.3320	0.4024	0.1313	0.1704
EmbeddingGemma	0.4053	0.2446	0.3100	0.3956	0.1562	0.3095
SigLIP 2	0.1658	0.0905	0.2160	0.2813	0.0938	0.2323

for these queries is $20/|G(q)|$ instead of 1, as described in Section 4.4.2. Table 7 therefore reports the values for the two groups separately. Precision is higher in the group with more than 20 notations for every method, by a factor between 1.94 for SigLIP 2 and 3.28 for BM25. Recall and MAP are higher in the group with 20 notations or fewer. The order of the methods differs between the two groups. EmbeddingGemma attains the highest precision value in the group with 20 or fewer notations, with a value of 0.1958, whereas BM25 attains the highest precision in the group with 20 or more notations with 0.4770.

Table 7: Mean values by the number of relevant notations per query

Method	≤ 20 Notations (n = 24))			≥ 20 Notations (n = 28))		
	Precision	Recall	MAP	Precision	Recall	MAP
BM25	0.1452	0.3268	0.1999	0.4770	0.1855	0.1513
SBERT	0.1792	0.4332	0.2851	0.4571	0.1931	0.1558
EmbeddingGemma	0.1958	0.4930	0.3332	0.4286	0.1851	0.1264
SigLIP 2	0.1188	0.3339	0.2068	0.2304	0.0928	0.0643

R-precision and MAP order the methods as recall does rather than as precision does. R-precision reaches 0.2637 for EmbeddingGemma, 0.2455 for SBERT, 0.2082 for BM25 and 0.1466 for SigLIP 2. MAP produces the same order with 0.2218, 0.2155, 0.1737 and 0.1301. The position of BM25 therefore differs from the one it takes on precision. The difference between BM25 and EmbeddingGemma amounts to 0.0481 on MAP and to 0.0555 on R-precision. Between EmbeddingGemma and SBERT it amounts to 0.0064 on MAP and to 0.0182 on R-precision, in favour of EmbeddingGemma in both cases, whereas precision favours SBERT. MAP remains below 0.4 for 45 of the 52 queries for BM25, for 42 for SBERT, for 43 for EmbeddingGemma and for 48 for SigLIP 2. The ten highest MAP values of the evaluation belong to queries whose ground truth comprises 13 notations or fewer.

The two directions of Wu-Palmer similarity yield different values and are reported separately before the mean. In the $G \rightarrow P$ direction the four values are close to each other, with 0.6228 for EmbeddingGemma, 0.6123 for SBERT, 0.5771 for BM25 and 0.5753 for SigLIP 2. The distance between the highest and the lowest value amounts to 0.0475. In

the $P \rightarrow G$ direction the same distance amounts to 0.2547, with 0.7290 for SBERT and 0.7215 for EmbeddingGemma against 0.6140 for BM25 and 0.4744 for SigLIP 2. The mean of the two directions gives 0.6721 for EmbeddingGemma, 0.6707 for SBERT, 0.5955 for BM25 and 0.5248 for SigLIP 2. The two leading methods keep the positions they take on precision and recall, but their difference decreases to 0.0015. BM25 remains third and SigLIP 2 fourth.

The two directions are not symmetric, and the sign of the difference is not the same for all four methods. For BM25, SBERT and EmbeddingGemma the $P \rightarrow G$ direction yields the higher of the two values, by 0.0370, 0.1167 and 0.0987 respectively. For SigLIP 2 it yields the lower one, by 0.1010. Counted per query, the $P \rightarrow G$ value is the higher one in 26 of the 52 queries for BM25, in 36 for SBERT and in 30 for EmbeddingGemma, but only in 22 for SigLIP 2. SigLIP 2 is therefore the only method whose retrieved notations are on average further from the ground truth than the ground truth is from them. A Wu-Palmer mean of 0.0000 occurs only for BM25, and only on “fireman”, “chandelier”, “Jupyter” and “commanding gesture”. The first three correspond to the empty result lists of Table 5. For the fourth, the nine retrieved notations share no common ancestor with any notation of the ground truth.

The Wu-Palmer values exceed precision, recall and the F_1 -score for every method. The difference between the Wu-Palmer mean and the F_1 -score amounts to 0.3748 for BM25, 0.4217 for SBERT, 0.4183 for EmbeddingGemma and 0.3828 for SigLIP 2. The same pattern appears on the queries for which a method records a precision of 0.0000. Restricted to these queries, the Wu-Palmer mean still reaches 0.2401 for BM25 over its nine such queries, 0.4557 for SBERT over five, 0.4406 for EmbeddingGemma over two and 0.3484 for SigLIP 2 over eight.

The mean values do not show how the four methods differ on individual queries. Two queries receive a precision of 0.0000 from all four methods: “Jesus with beard” and “Saint John”. Under Wu-Palmer similarity, the two queries behave differently. For “Jesus with beard”, the values are 0.6883 for SBERT and 0.6362 for SigLIP 2, compared with 0.2702 for BM25 and 0.2584 for EmbeddingGemma. For “Saint John” they are 0.7873 for SBERT, 0.6228 for EmbeddingGemma, 0.5087 for BM25 and 0.2660 for SigLIP 2. The ground truth of “Jesus with beard” consists of 18 notations under 11D and none of their labels contain the word “beard”. SBERT returned 18 notations of the group 11DD, which refers to the beardless Christ. For “Saint John”, all 20 notations returned by SBERT are of the form 11H(JOHN)...), as are 22 of the 26 notations of the ground truth, but the two sets do not overlap.

For seven queries, at least one of the embedding-based methods reaches a precision above 0.0000, while BM25 reaches 0.0000. These queries are “fireman”, “chandelier”, “Jupyter”, “commanding gesture”, “saint”, “officer riding a horse” and “Jesus without beard”. Three

of them are the empty BM25 lists. The largest single difference occurs on “Jesus without beard”, where SBERT and SigLIP 2 both reach 0.9000. Among the queries for which BM25 did return notations, the largest difference occurs on “commanding gesture”, where EmbeddingGemma reaches 0.5500 against 0.0000 for BM25 over nine returned notations. There is no query for which BM25 reaches a precision above 0.0000, while all three embedding-based methods remain at 0.000. Under a weaker criterion, seven queries remain in which the precision of BM25 is at least twice the highest precision reached by an embedding-based method. These are “hatred” with 1.000 against 0.2000, “castle” with 0.7857 against 0.3000, “lion” with 0.6000 against 0.3000, “dragon” with 0.4000 against 0.2000, “young girl” and “aggressive” with 0.3000 against 0.1500 each, and “portrait of a girl” with 0.1500 against 0.0500. In all seven, the words of the query appear in the labels of the notations BM25 returned.

The pair “Jupiter” and its misspelled variant “Jupyter”, described in Section 4.2, separates the four methods more than any other pair of queries. Both queries carry the same ground truth of 118 notations. For “Jupiter”, the precision values are 0.9000 for BM25, 0.8000 for SBERT, 0.8000 for EmbeddingGemma and 0.0000 for SigLIP 2. On “Jupyter” only EmbeddingGemma keeps a value above zero with 0.4500, while BM25, SBERT and SigLIP 2 all fall to 0.0000. EmbeddingGemma returns eight of the same 20 notations for both queries, whereas BM25 and SBERT return none in common.

SigLIP 2 records the lowest mean on all eight metrics, but it does not remain below the other three methods on every query. On nine queries it reaches the highest value of the four methods, alone or shared, and on nine queries, the highest recall. The same applies to eight queries on average precision and to twelve in the $G \rightarrow P$ direction of the Wu-Palmer similarity. Its highest average precision in the evaluation, 0.8667 on “saint holding a monstrance”, is also the highest value any method reaches on that query, where SBERT reaches 0.000. On “making lace” and “elderly woman” it reaches 0.7321 and 0.4874 against 0.2500 and 0.1111 for BM25. Its mean in the $G \rightarrow P$ direction, 0.5753, remains 0.0017 below that of BM25, whereas the distance to BM25 in the $P \rightarrow G$ direction amounts to 0.1397.

Five of the queries described above are listed in Table 8 with their precision and their Wu-Palmer mean.

6 Discussion

The aim of this thesis was to find out which type of text representation is most suitable for matching short textual queries to Iconclass labels. The two embedding models that were trained on text alone, SBERT and EmbeddingsGemma, find more of the relevant

Table 8: Selected queries with precision and Wu-Palmer mean per method

Query	$ \mathbf{G}(\mathbf{q}) $	BM25	SBERT	EmbeddingGemma	SigLIP 2
Precision					
“Saint John”	26	0.0000	0.0000	0.0000	0.0000
“fireman”	8	0.0000	0.3000	0.4000	0.1500
“hatred”	29	1.0000	0.2000	0.1500	0.0500
“Jupyter”	118	0.0000	0.0000	0.4500	0.0000
“saint holding a monstrance”	3	0.1000	0.0000	0.1000	0.1500
Wu-Palmer mean					
“Saint John”	26	0.5087	0.7873	0.6228	0.2660
“fireman”	8	0.0000	0.7711	0.8311	0.6076
“hatred”	29	0.5511	0.4558	0.4690	0.1905
“Jupyter”	118	0.0000	0.1479	0.6019	0.3259
“saint holding a monstrance”	3	0.6127	0.4730	0.7385	0.8156

notations than the lexical baseline and place them higher, but they do not reach a higher precision. The text encoder of SigLIP 2 has the lowest mean value on all eight metrics, and yet is the best of the four methods on several individual queries. Under Wu-Palmer similarity, all four methods reach much higher values than under precision, recall, and the F_1 -score. Taken together, these observations do not point to one method that is simply better than the others, but to four separate effects: the ranking of the methods depends on which aspect of retrieval quality is measured, the precision of BM25 is partly a result of the notations it does not return, SigLIP 2 text encoder differs from the other methods in the kind of error it makes rather than in how often it makes one, and the binary view of relevance hides how close the methods come to the intended part of the hierarchy.

The first effect matters most for the comparison. Precision places SBERT first and BM25 second, while recall, R-Precision and MAP place EmbeddingGemma first, SBERT second and BM25 third (see Table 4). This is not a contradiction, because the two groups of metrics describe different things. Precision describes how reliable a list of candidates is once it is shown. The other metrics describe whether the relevant part of the classification is found at all, and how early it appears. In the task considered here, a cataloger receives a small number of suggestions and decides which of them to use. A wrong suggestion costs a moment of reading, but a category that never appears cannot be chosen. For this task, then, recall, R-Precision and MAP can be considered the more informative metrics. The advantages of SBERT and EmbeddingGemma lie in finding the relevant notations at all, and not in the reliability of the list they return.

This shortfall in precision is also less serious than it first appears. As described in Section 4.3.1, BM25 only scores notations that share at least one term with the query. It consequently returns fewer than 20 notations for seven queries, and nothing at all for

three of them (see Table 5). Its precision is not measured on the same kind of list as the precision of the other methods, which always return 20 candidates. Its highest precision values come from these short lists. At first sight the precision values suggest that term matching is the more accurate representation. On closer inspection they suggest something narrower. Term overlap is a strong signal when the query terms appear in a label. Its precision stays high because the method returns nothing when they do not, instead of filling the list with arbitrary candidates. The empty result lists are the price of this reliability, and a mean value does not show them.

Whether the query terms appear in the relevant labels is a property of the query, not of the method, and the results show that this varies with the length of the query. On the queries with one word, BM25 reaches the highest precision of all four methods. On the queries of two or three words, both SBERT and EmbeddingGemma are ahead of it (see Table 6). Single terms such as “castle” or “lion” are often words that the vocabulary itself uses. Query and label then share the same terms, and a semantic representation has little to add. As soon as a query describes a subject instead of naming it, this overlap disappears. The gap between the wording of the cataloger and the wording of Iconclass, described in Section 1, then becomes visible in the results. This is the situation in which a query such as “woman holding a baby” has to reach a label such as “Madonna and the Child”, and it is also the situation in which SBERT and EmbeddingGemma gain their advantage.

The advantage does not extend to the longest queries, where all four methods reach their lowest precision values. One possible reason is that these queries do not add information but combine it. A query such as “officer riding a horse” describes a relation between several concepts, while the embedding models reduce the whole text to a single vector, as described in Section 3.1.2. Such a representation keeps the individual concepts, but not the way they are connected. This explanation cannot be confirmed here, since the results do not show whether the drop comes from combining several concepts or simply from longer queries. For BM25 the reason is different, since a longer query gives more terms the chance to match an unrelated label.

The pair “Jupiter” and its misspelled variant “Jupyter” shows a limit of a different kind. For the correct spelling, BM25, SBERT and EmbeddingGemma reach high precision. For the misspelled variant, only EmbeddingGemma still returns relevant notations, and it is the only one of these three methods that returns any of the same notations for both spellings. The failure of BM25 is expected, because the changed string shares no term with any label. The failure of SBERT is more informative, because it rules out the most obvious explanation. Both models divide a less frequent word into several parts, as described in Section 3.1.2, so this alone is not enough to absorb a spelling error. The two differ in size and in the type of model each is derived from, and the results do not

allow these factors to be separated. The observation is therefore a difference between two specific models rather than a property of embedding-based methods in general.

How the SigLIP 2 text encoder differs from the other methods matters more than the size of the difference. Its mean values are the lowest throughout, but the two directions of Wu-Palmer similarity separate it from the other methods in a way that precision and recall do not. For the other three methods, the direction from retrieved notations to the ground truth gives the higher of the two values. For SigLIP 2, it gives the lower one, and this also holds for most of its individual queries (see Section 5). The notations it retrieves are on average further away from the ground truth than the ground truth is from them. This is a different situation from being less accurate. When the other methods make a mistake, they tend to stay in the region of the hierarchy in which the ground truth lies, while SigLIP 2 tends to retrieve notations from other parts of it. This fits the way its embedding space was learned. As described in Section 3.2.4, its training objective is defined over image-text pairs rather than pairs of texts, so distances in its embedding space reflects which descriptions belong to similar images. Iconclass, however, groups notations by iconographic meaning and not by visual appearance. A representation built for image search is likely to place labels close together that Iconclass keeps apart, and the asymmetry observed here is consistent with this.

At the same time, the encoder does not fall behind on every query, which makes the difference one of type rather than of level. On several queries, it reaches the highest precision, recall or average precision of all four methods (see Section 5). The queries on which it succeeds tend to describe a scene that can be depicted, such as “making lace” or “saint holding a monstrance”. This is what a representation trained on captions can be expected to capture. Its strengths therefore lie elsewhere than those of the text-only models, which makes it complementary to them rather than inferior.

Precision, recall and the F_1 -score treat relevance as a binary property, and Wu-Palmer similarity shows how much this leaves out. For every method, the Wu-Palmer mean is clearly higher than these three values. The same is true for the queries that receive a precision of zero. Two queries make the reason visible. For “Saint John”, none of the methods retrieves a relevant notation, but every notation that SBERT returns belongs to the same group of the hierarchy as the greatest majority of the notations of the ground truth. The two sets simply don’t overlap. For “Jesus with beard”, where no ground truth label contains the word “beard” SBERT returns notations of the group that stands for the beardless Christ. A binary judgment counts both cases as complete failures, although the method identified the correct person in the first case and the correct iconographic type in the second. This is the difficulty described in Section 2.3. Much of what these metrics record as an error is not a failure to find the subject, but a failure to select the exact notation inside a region that was formed correctly.

These values have to be interpreted with care, because closeness in the hierarchy is not the same as relevance. A cataloguer who assigns a notation to an image needs the notation that applies, not one that is related to it. A high Wu-Palmer value therefore does not mean that most suggestions can be used. What the difference between the two groups of metrics shows is something about the evaluation rather than about the quality of the output. In an interface in which a person accepts or rejects suggestions, however, this limitation weighs less, since a notation from the right region gives a starting point in the hierarchy even when it is not the notation that will be assigned.

Wu-Palmer similarity also shows that the methods fail in different ways. A mean of zero occurs only for BM25, and only for four queries, three of which are the empty result lists. The failure of the lexical method is abrupt: it either finds the term or returns nothing usable. The failure of SBERT and EmbeddingGemma is gradual and stays near the intended region.

Considered as a whole, the results point to a connection between the type of query and the type of representation that suits it. Term overlap remains the most precise signal for single terms that appear in the Iconclass labels. Embedding models are needed as soon as a query describes rather than names a subject, and SBERT and EmbeddingGemma are also the only methods that stay close to the intended region when they are wrong. The results do not support a ranking between the two, since the differences are small and each leads on some of the metrics. This has a practical implication, because SBERT reaches these values with 22 million parameters while EmbeddingGemma has 308 million. For a service that has to embed a collection of this size, the smaller model is the more reasonable default. The complementary behaviour of the lexical and the embedding based methods also suggests combining them instead of choosing between them, since the queries on which BM25 is clearly ahead and those on which it returns nothing are largely different queries.

These conclusions hold within a number of limits. Only the first 20 retrieved notations were evaluated, and for more than half of the queries the ground truth contains more notations than that. The highest reachable recall is therefore lower than one for these queries, and the pattern by which precision is higher and recall lower for them (see Table 7) follows from this cutoff rather than from the methods. The ground truth also consists of 52 queries formulated for this evaluation with binary relevance judgments, and it was defined on the filtered collection instead of the full Iconclass pool. A query set of this size cannot represent the whole range of ways in which catalogers search. All methods were used in their default configuration, without fine-tuning, query expansion or re-ranking, so the results describe these representations in their default form rather than the best performance that could be reached with them. Only English labels and textual queries were considered, and for SigLIP 2 only the text encoder. Finally, the Wu-Palmer values

depend on how the hierarchy was modeled in Section 4.1.3, where notations from different top-level categories share no common ancestor. The values can therefore be compared across the four methods evaluated here, but not with values computed on a hierarchy that is modeled differently.

Within these limits, the evaluation supports a consistent picture. The text-only embedding models find the relevant region of Iconclass more reliably than term overlap for the descriptive queries that motivate this task, while term overlap remains the stronger signal where query and label share the same wording. A text encoder trained on captions differs from text-only models in the kind of error it makes, rather than only in how often it makes one. A measure that takes the hierarchy into account is necessary to see any of this.

7 Conclusion

Iconclass organises the subjects of images in a large hierarchy, in which each notation carries a short label, and finding the label that fits a query is a retrieval task. Embedding-based methods are known to work well for such tasks in general, but Iconclass differs from the settings in which this was shown, since its labels are short and domain-specific and belong to a large hierarchical structure. It was therefore not clear which type of text representation is suited to this task, or where the methods differ from one another. To answer this, BM25, SBERT, EmbeddingGemma and the text encoder of SigLIP2 were applied to the same queries and the same collection, and their results were compared against a manually built ground truth.

The comparison did not point to one method that is better than the others. The two embedding models trained on text alone found more of the relevant notations than BM25 and placed them higher in the list, but they did not return more reliable lists. What decided the outcome was the query rather than the method. Where a query used a word that also appears in the labels, term matching remained the strongest signal. Where a query described a subject in different words, the embedding models were the more dependable route to the right labels. SBERT and EmbeddingGemma behaved so similarly that the differences between them do not justify preferring the larger one. The text encoder of SigLIP2 was the weakest on average, but it failed in a different way than the others, since its results tended to leave the part of the hierarchy in which the relevant notations lie. On queries that describe a scene as it would be pictured, it was nevertheless the best of the four methods.

Reading the results through the hierarchy changed what they mean. For every method, much of what the other metrics counted as an error was a near miss, in which the correct region of Iconclass was reached but not the exact notation. Binary relevance alone

therefore hides how the methods differ, even though a notation from the right region is still not the notation a user needs.

Taken together, this suggests that the methods should be combined rather than chosen between, since they serve different queries, and that the smaller of the two text models is sufficient. It also shows that an evaluation of retrieval in a classification system has to take its hierarchy into account.

These conclusions hold within the limits described in Section 6: a ground truth built by hand for a limited set of queries, labels in a single language, and models used in their default configuration. These limits also show where further work is needed. A larger ground truth with graded relevance would allow the ranking of the methods to be judged more finely, and an evaluation with the users of Iconclass would test the relevance decisions themselves. Since Iconclass is multilingual, the comparison should also be repeated in other languages. Beyond this, a combination of a lexical and an embedding-based method is worth building and testing, and the image side of SigLIP2, which was left aside here, would allow a search to start from the artwork itself.

References

- [1] ALABDULMOHSIN, I., ZHAI, X., KOLESNIKOV, A., AND BEYER, L. Getting vit in shape: Scaling laws for compute-optimal model design. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)* (2023).
- [2] BRANDHORST, H., AND POSTHUMUS, E. Exploring iconclass. a comprehensive system for cultural content classification.
- [3] DEVLIN, J., CHANG, M.-W., LEE, K., AND TOUTANOVA, K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (Minneapolis, Minnesota, June 2019), J. Burstein, C. Doran, and T. Solorio, Eds., Association for Computational Linguistics, pp. 4171–4186.
- [4] GOOGLE. Embeddinggemma model card, 2025. Accessed: 2026-07-08.
- [5] GOOGLE. siglip2-so400m-patch16-naflex. Model card, Hugging Face, 2025. accessed 31 August 2026.
- [6] HUGGING FACE. modeling_siglip2.py — transformers source code. https://github.com/huggingface/transformers/blob/main/src/transformers/models/siglip2/modeling_siglip2.py, 2025. Accessed: 2026-08-28.
- [7] HUGGING FACE. SigLIP2 — transformers documentation. https://huggingface.co/docs/transformers/en/model_doc/siglip2, 2025. Accessed: 2026-08-28.
- [8] KRANTZ, T., AND JONKER, A. What is ground truth?
- [9] KUDO, T., AND RICHARDSON, J. Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (Brussels, Belgium, 2018), Association for Computational Linguistics, pp. 66–71.
- [10] KUSUPATI, A., BHATT, G., REGE, A., WALLINGFORD, M., SINHA, A., RAMANUJAN, V., HOWARD-SNYDER, W., CHEN, K., KAKADE, S., JAIN, P., AND FARHADI, A. Matryoshka representation learning, 2024.
- [11] LIN, Z., FENG, M., DOS SANTOS, C. N., YU, M., XIANG, B., ZHOU, B., AND BENGIO, Y. A structured self-attentive sentence embedding, 2017.
- [12] MACAVANEY, S. *Deep Learning for Search*. Manning, 2020. See Chapter 6: Document embeddings for rankings and recommendations.

- [13] MANNING, C. D., RAGHAVAN, P., AND SCHÜTZE, H. *Language models for information retrieval*. Cambridge University Press, 2008.
- [14] MIHALCEA, R., CORLEY, C., AND STRAPPARAVA, C. Corpus-based and knowledge-based measures of text semantic similarity. In *Proceedings of the 21st National Conference on Artificial Intelligence* (Boston, Massachusetts, USA, 2006), AAAI Press, pp. 775–780.
- [15] MITCHELL, J., AND LAPATA, M. Composition in distributional models of semantics. *Cognitive Science* 34, 8 (2010), 1388–1429.
- [16] MITRA, B., AND CRASWELL, N. Neural models for information retrieval. *arXiv preprint arXiv:1705.01509* (2017).
- [17] MITRA, B., AND CRASWELL, N. An introduction to neural information retrieval. *Foundations and Trends in Information Retrieval* 13, 1 (2018), 1–126.
- [18] MULLER, M., WOLF, C. T., ANDRES, J., DESMOND, M., JOSHI, N. N., ASHKTORAB, Z., SHARMA, A., BRIMIJOIN, K., PAN, Q., DUESTERWALD, E., AND DUGAN, C. Designing ground truth and the social life of labels. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems* (New York, NY, USA, 2021), CHI '21, Association for Computing Machinery.
- [19] PESQUITA, C., FARIA, D., BASTOS, H., FERREIRA, A. E. N., FALCÃO, A. O., AND COUTO, F. M. Metrics for GO based protein semantic similarity: a systematic evaluation. *BMC Bioinformatics* 9, Suppl 5 (2008), S4.
- [20] POSTHUMUS, E., AND BRANDHORST, J. ICONCLASS. <https://iconclass.org>. Accessed: 2026-09-01.
- [21] REDDY, J. R., AND MISHRA, S. K. Understanding vector embeddings in AI search: A technical deep dive. *International Journal of Computer Engineering and Technology* 15, 6 (2024), 147–160.
- [22] REIMERS, N., AND GUREVYCH, I. Retrieve & re-rank pipeline — sentence transformers documentation. https://www.sbert.net/examples/sentence_transformer/applications/retrieve_rerank/README.html. Accessed: 2026-07-09.
- [23] REIMERS, N., AND GUREVYCH, I. Semantic search — sentence transformers documentation. https://sbert.net/examples/sentence_transformer/applications/semantic-search/README.html. Accessed: 2026-07-09.
- [24] REIMERS, N., AND GUREVYCH, I. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on*

- Natural Language Processing (EMNLP-IJCNLP)* (Hong Kong, China, Nov. 2019), K. Inui, J. Jiang, V. Ng, and X. Wan, Eds., Association for Computational Linguistics, pp. 3982–3992.
- [25] ROBERTSON, S., AND ZARAGOZA, H. The probabilistic relevance framework: Bm25 and beyond. *Found. Trends Inf. Retr.* 3, 4 (Apr. 2009), 333–389.
- [26] SALTON, G., AND BUCKLEY, C. Term-weighting approaches in automatic text retrieval. *Inf. Process. Manag.* 24 (1988), 513–523.
- [27] SCHLICKER, A., DOMINGUES, F. S., RAHNENFÜHRER, J., AND LENGAUER, T. A new measure for functional similarity of gene products based on gene ontology. *BMC Bioinformatics* 7, 302 (2006).
- [28] SENNRICH, R., HADDOW, B., AND BIRCH, A. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Berlin, Germany, 2016), Association for Computational Linguistics, pp. 1715–1725.
- [29] SENTENCE-TRANSFORMERS. all-minilm-l6-v2 model card. <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>, 2025. Accessed: 2026-08-25.
- [30] SPÄRCK JONES, K. A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation* 60, 5 (10 2004), 493–502.
- [31] STANFORD UNIVERSITY. Cs124 information retrieval group work extra handout. http://web.stanford.edu/class/cs124/lec/CS124_IR_Handout.pdf, 2016. Accessed: 2026-07-09.
- [32] STOJANOVSKI, E., NAKOV, P., ET AL. Text similarity estimation based on word embeddings and conventional NLP features. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics* (2019).
- [33] TAY, Y., DEHGHANI, M., BAHRI, D., AND METZLER, D. Efficient transformers: A survey, 2022.
- [34] TSCHANNEN, M., GRITSENKO, A., WANG, X., NAEEM, M. F., ALABDULMOHSIN, I., PARTHASARATHY, N., EVANS, T., BEYER, L., XIA, Y., MUSTAFA, B., HÉNAFF, O., HARMSSEN, J., STEINER, A., AND ZHAI, X. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint* (2025).
- [35] VERA, H. S., ET AL. Embeddinggemma: Powerful and lightweight text representations, 2025.

-
- [36] WANG, W., WEI, F., DONG, L., BAO, H., YANG, N., AND ZHOU, M. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems* (2020), vol. 33, Curran Associates, Inc.
- [37] WU, Z., AND PALMER, M. Verb semantics and lexical selection. In *Proceedings of the 32nd Annual Meeting of the Association for Computational Linguistics* (Las Cruces, New Mexico, USA, June 1994), Association for Computational Linguistics, pp. 133–138.
- [38] ZHAI, X., MUSTAFA, B., KOLESNIKOV, A., AND BEYER, L. Sigmoid loss for language image pre-training, 2023.

Assertion

Ich versichere wahrheitsgemäß, die Arbeit selbstständig verfasst, alle benutzten Hilfsmittel vollständig und genau angegeben und alles kenntlich gemacht zu haben, was aus Arbeiten anderer unverändert oder mit Abänderungen entnommen wurde sowie die Satzung des KIT zur Sicherung guter wissenschaftlicher Praxis in der jeweils gültigen Fassung beachtet zu haben.

Karlsruhe, September 4, 2026

Beril Tezel

A handwritten signature in blue ink, appearing to read 'Beril Tezel', with a horizontal line drawn across the middle of the letters.